AirMouse

Autonomous GPS-denied indoor search, mapping and survivor localisation drone

NIDAR Track 1 · Drone Innovation

Document	Project overview, tech stack, status, and how to install and run it
Repository	https://github.com/harshkabir007/airmouse
Version	main @ b1246f5 · 11 October 2026
Status	Simulation: full autonomous mission working. Hardware: not started.

Contents

1. Project overview
2. System architecture
3. Tech stack
4. Software components
5. Frames, map and rescue grid
6. What works today (status)
7. Simulation results
8. Problems found and fixed during bring-up
9. How to install (new machine)
10. How to run the simulation
11. Using the ground control station
12. Running on the real drone (Raspberry Pi 5)
13. Configuration reference
14. Troubleshooting
15. Repository layout
16. Roadmap and next steps

Appendix: command cheat sheet

1. Project overview

NIDAR AirMouse simulates a collapsed building: a covered, maze-like arena of up to 15 m × 15 m with 1 m corridors, 2 m × 2 m rooms and 8 ft (2.44 m) walls. A drone has to enter through one entry point with **no GPS**, explore the maze, find up to **6 survivors**, build a **live 2D map** on the ground control station (GCS), tag the **grid cell** of each survivor, and leave through the same point within **30 minutes**. The operator may only start the mission and abort it. Everything else must be autonomous.

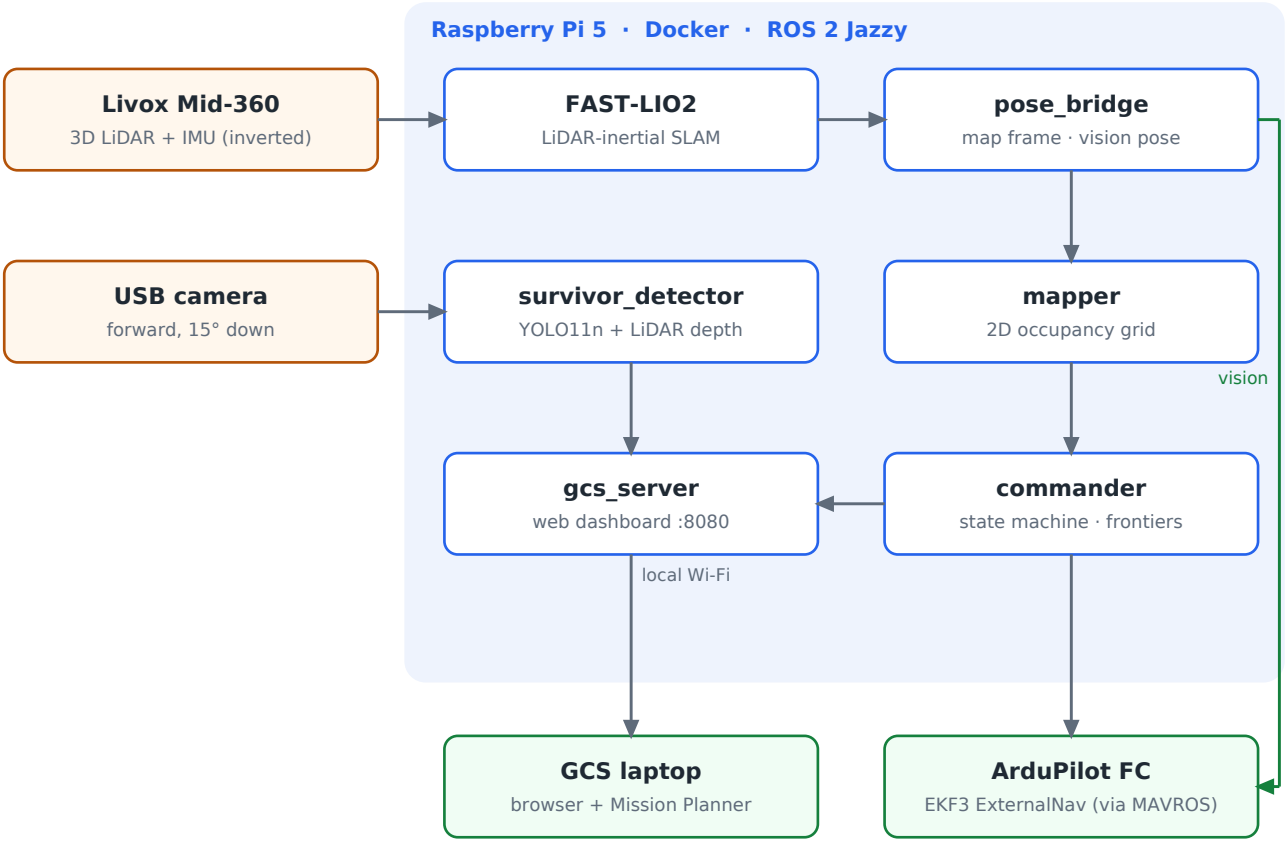
AirMouse is our answer. The drone localises itself with a 3D LiDAR and IMU (LiDAR SLAM), and feeds that position to the flight controller in place of GPS. It explores with a frontier planner, detects people with a neural network on a camera, places them on the map using the LiDAR, and streams the map, survivor tags and video to a web dashboard over a local Wi-Fi link.

Design goals

- **Same code in simulation and on the drone.** Gazebo and ArduPilot SITL replace the sensors and the flight controller; every autonomy node is unchanged.
- **Portable.** Everything runs in Docker. A new laptop needs only Docker (plus WSL2 on Windows) and a `git clone`.
- **Safe by default.** Every failsafe ends in LAND. The drone never does a straight-line "return home" that would fly through walls.
- **Rule-compliant.** No GPS, no internet, no manual control: the operator only has START, RECALL and ABORT.

2. System architecture

The drone carries a Raspberry Pi 5 running ROS 2 in Docker. The LiDAR data goes through FAST-LIO2 to get the drone pose. That pose goes to the flight controller (as an external navigation source for its EKF3 position estimator) and to the mapping, planning and detection nodes. The web GCS is served directly from the drone.



Data flow of the autonomy stack on the drone. In simulation, Gazebo provides the LiDAR, IMU and camera, and ArduPilot SITL replaces the flight controller.

Mission state machine

```
IDLE -> PREFLIGHT -> ARMING -> TAKEOFF -> EXPLORE -> RETURN -> LANDING -> DONE
                         any flying state -> LANDING   (failsafe or ABORT)
                         EXPLORE          -> RETURN    (RECALL, time budget, low battery)
```

- **PREFLIGHT**: wait for the flight-controller link and healthy SLAM, set the EKF origin, switch to GUIDED mode.
- **EXPLORE**: every second, pick the best frontier (boundary between known free space and unknown space inside the arena), plan a path on the map, and follow it. At each frontier the drone does a 360° yaw sweep so the camera sees the room.
- **RETURN**: plan a path back to the entry point through the mapped corridors, then land.

3. Tech stack

Layer	Choice	Why
Flight controller firmware	ArduPilot ArduCopter 4.6.3	Mission Planner support; mature EKF3 "ExternalNav" mode that accepts a SLAM pose instead of GPS.
SLAM	FAST-LIO2 (hku-mars, ROS 2 branch)	Tightly coupled LiDAR + IMU, robust to fast motion, runs well on ARM. Standard choice for Livox drones.
LiDAR	Livox Mid-360, mounted upside-down	360° × 59° field of view, built-in IMU, 265 g. Inverted, it sees the floor, so it also sees survivors lying down.
Companion computer	Raspberry Pi 5 (16 GB)	Enough CPU for FAST-LIO, mapping, planning and a small detector at a few Hz.
Middleware	ROS 2 Jazzy (Ubuntu 24.04)	Long-term support release; native Gazebo Harmonic pairing.
FC link	MAVROS (MAVLink)	Vision pose in, setpoints / mode / arming out.
Simulation	Gazebo Harmonic + ArduPilot SITL	SITL runs the real ArduPilot code, so EKF and failsafe behaviour transfers to the real drone.
Person detection	YOLO11n (Ultralytics), CPU	Small and fast; can be exported to NCNN for the Pi.
Autonomy nodes	Python 3 (rclpy, NumPy, SciPy, OpenCV)	Easy to read and change; heavy lifting is vectorised.
Ground control	Web dashboard (no internet needed) + Mission Planner	Live map, grid, survivor tags, video and buttons in any browser.
Packaging	Docker + docker compose + VS Code dev container	Identical environment on every laptop and on the Pi.
Dev environment	Windows 11 + WSL2 (Ubuntu 24.04), or native Linux	Tested on Windows 11 with WSL2.

4. Software components

All AirMouse code is in `ros2_ws/src/`, split into three ROS 2 packages.

Node (package)	What it does	Main topics
pose_bridge (airmouse)	Turns FAST-LIO output into the drone-centred "map" frame (handles the upside-down LiDAR). Sends the pose to ArduPilot. Flags SLAM as unhealthy if it goes stale or jumps.	in: /Odometry, /cloud_registered out: /airmouse/odom, /airmouse/cloud, /mavros/vision_pose/pose, /airmouse/slam_ok
mapper (airmouse)	Builds the live 2D occupancy grid (5 cm cells) from the 3D scans: slices a height band, marks walls, ray-traces free space, locks confirmed walls.	out: /airmouse/map
commander (airmouse)	Mission state machine, frontier exploration, path planning, carrot path following, safe yaw handling, return to exit, all failsafes.	out: /mavros/setpoint_position/local, /airmouse/status, /airmouse/path
survivor_detector (airmouse)	YOLO person detection on the camera. Localises each detection with LiDAR points inside the box (or a floor-plane ray), fuses repeated sightings, tags the grid cell.	out: /airmouse/survivors, /airmouse/detections
gcs_server (airmouse)	Web dashboard and HTTP API: map image, state JSON, MJPEG video, START / RECALL / ABORT.	http://<drone>:8080
lidar_filter (airmouse_sim)	Simulation only: removes "no hit" points and adds exact point timing to Gazebo LiDAR scans so FAST-LIO works correctly.	in: /lidar/points out: /lidar/points_clean
maze_gen, drone_model (airmouse_sim)	Turn an ASCII maze drawing into a Gazebo world, and add the LiDAR, IMU and camera to ArduPilot's simulated quad.	-
launch files (airmouse_bringup)	sim.launch.py (whole simulation), robot.launch.py (real drone), core.launch.py (shared nodes). All parameters live in config/.	-

5. Frames, map and rescue grid

- **Map frame** = the drone at take-off. +x points into the maze from the entry (shown as "up" on the GCS), +y to the left, z up. ArduPilot's local frame is the same frame.
- **Rescue grid** (configurable): the origin is the left-hand corner of the entry wall, seen when facing into the maze. Columns are letters (A, B, C …) left to right; rows are numbers (1, 2, 3 …) from the entry wall inward. Default cell size is 1 m. So "A1" is the cell at the entry wall on your left.
- On competition day, set `arena.origin_x / origin_y` from where the launch box is, and change the cell size and labelling to match the organisers' grid.

6. What works today (status)

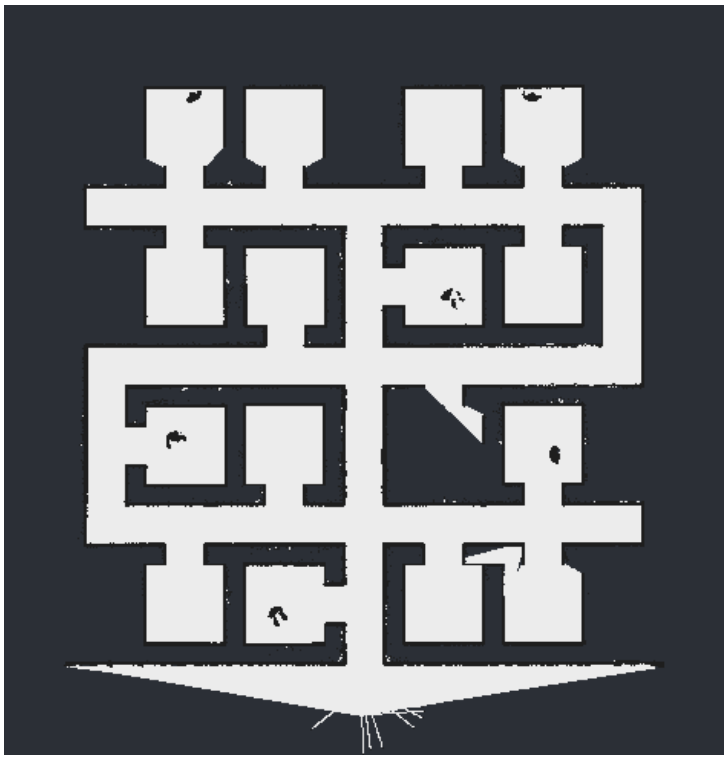
Summary. The complete mission works autonomously in simulation: take-off, exploration of the whole maze, live map, survivor tagging, return and landing at the exit. It has been tested on one laptop with one maze layout and is not yet reliable enough to call finished. Nothing has been tested on real hardware yet.

Feature	Status	Notes
Docker simulation image (laptop, amd64)	**Working**	Builds from scratch; about 12 GB.
Gazebo maze world from ASCII layout	**Working**	Includes 6 survivor models.
Simulated drone with LiDAR, IMU, camera	**Working**	ArduPilot iris + injected sensors.
FAST-LIO2 SLAM in simulation	**Working**	Stable over a 13.5 min flight after the timing fix.
ArduPilot EKF3 using SLAM instead of GPS	**Working**	Vision pose and EKF agree to a few cm.
Autonomous preflight, arming, take-off	**Working**	GUIDED mode, 1.2 m altitude.
Live 2D occupancy map	**Working**	All 16 rooms and corridors clearly visible.
Frontier exploration of the full maze	**Working**	Covered the whole maze; some areas visited twice. One frontier was skipped after a timeout.
Survivor detection and grid tagging	**Partial**	5/6 in the best complete run, 6/6 in an earlier run; positions within about 0.3 m.
Return to exit and landing	**Working**	Landed within 7 cm of the start point.
Failsafe: SLAM lost → land	**Verified in sim**	Triggered during earlier crashes.
Failsafe: geofence → land	**Verified in sim**	Triggered when SLAM diverged.
Failsafe: low battery, time budget, abort, recall	**Implemented, untested**	Code paths exist; not yet exercised in a test.
Web GCS dashboard	**Partial**	Server, map and state API verified over HTTP; the browser page and video stream still need a visual check.
Mission Planner connection	**Implemented, untested**	TCP 5762 in sim / UDP 14550 on drone.
Robot image on Raspberry Pi 5 (arm64)	**Untested**	Same Dockerfile; never built on a Pi.
Livox Mid-360 driver, USB camera on the Pi	**Untested**	Configs written; no hardware yet.
Real flight	**Not started**	Phases 6–8 of the implementation plan.

7. Simulation results

Best complete run (maze_01, default 1.3× maze scale, Windows 11 laptop with Intel Iris Xe graphics, simulation running at about 0.7× real time):

Metric	Result
Mission outcome	Fully autonomous: START → explore → return → landed at exit
Mission time (sim time)	13.5 min of the 30 min limit (exploration done at 12.3 min)
Area mapped	about 197 m² of free space inside the arena
Survivors tagged	5 of 6, all in the correct grid cell (F2, C6, J10, L15, M6)
Survivor position error	about 0.1–0.3 m against Gazebo ground truth
Landing accuracy	7 cm from the take-off point
Crashes	None in this run (earlier runs crashed; see section 8)



The live 2D map exactly as the GCS showed it at the end of that run. White is free space, dark grey is unknown, black is walls. The entry is at the bottom. Small dark blobs inside rooms are the survivors.

8. Problems found and fixed during bring-up

Every crash in testing was traced in flight logs to a root cause before fixing it. These are worth knowing because several will matter on the real drone too.

Problem	Root cause	Fix
FAST-LIO and Livox driver did not compile	Code written for older ROS 2 / C++14	Build script switches to the Jazzy code path and C++17; versions pinned.
Gazebo picked the wrong drone model	model://airmouse matched the repo folder	Generated model renamed to airmouse_drone.
FAST-LIO never started (sim)	Gazebo reports missed beams as "inf" points	lidar_filter node removes them.
SLAM froze right after take-off (sim)	No per-point time in Gazebo scans; FAST-LIO's guess ran away	lidar_filter adds exact capture times.
Drone flipped while turning in a doorway	Fast 90° yaw in a 1 m door caused drift into the wall	Ramped yaw; turn on the spot only where clearance allows.
Drone chased "frontiers" on walls	Unseen insides of thick walls look like unexplored space	Frontier cells must not touch walls.
Drone hit door frames	Simulated iris is about 1.5× wider than our airframe	Sim maze scaled 1.3× (maze_scale) to keep the right proportions.
Drone clipped wall corners	Grazing LiDAR rays slowly erased thin corners from the map	Confirmed wall cells are locked.
Drone drifted off its path through doors	Far waypoints let small errors carry diagonally	Carrot-following along the planned segment.
Same survivor tagged twice	First far-away sightings landed >1 m apart	Tracks fused when they converge.
Commander kept "exploring" after a crash	Flight controller disarmed but mode stayed GUIDED	Abort when disarmed in flight.
Simulation ran at 10% speed	WSL GPU translation is slow for sensor rendering	Software rendering (llvmpipe) by default: about 0.7× real time.

9. How to install (new machine)

You need about 30 GB of free disk, 16 GB of RAM (recommended), and an internet connection for the first build (30–60 min). After that, the simulation runs offline except for the first download of the survivor models.

Windows 11 (tested)

1. Open **PowerShell as Administrator** and install WSL2 with Ubuntu 24.04. Reboot if asked.

```
wsl --install -d Ubuntu-24.04
```

2. Give WSL enough memory for the builds. Create the file `C:\Users\<you>\.wslconfig` with this content, then run `wsl --shutdown`.

```
[wsl2]
memory=10GB
swap=8GB
processors=12
```

3. Open the **Ubuntu** app from the Start menu. On first launch, create a Linux username and password.
4. Install Docker inside Ubuntu, then close and reopen the Ubuntu window:

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
```

5. Clone the project **inside the Linux home folder**, not under /mnt/c or OneDrive (a Windows path makes builds slow and breaks file links):

```
cd ~
git clone https://github.com/harshkabir007/airmouse
cd airmouse
```

6. Build the simulation image (30–60 minutes the first time):

```
docker compose build sim
```

Linux (Ubuntu 22.04 / 24.04)

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER   # then log out and in
git clone https://github.com/harshkabir007/airmouse && cd airmouse
docker compose build sim
```

> **Sharing a ready-made image.** To skip the long build on another laptop, export it once with `docker save airmouse:sim | gzip > airmouse_sim.tar.gz` and import it there with `docker load < airmouse_sim.tar.gz`.

10. How to run the simulation

> **Windows:** keep an Ubuntu window open the whole time. WSL shuts Ubuntu down, and the simulation with it, when no Ubuntu window is attached.

1. **Start the container** (from the airmouse folder in Ubuntu). On Windows use the WSL override, which enables Gazebo windows and fast rendering:

```
# Windows (WSL)
docker compose -f compose.yaml -f docker/compose.wsl.yaml up -d sim

# Linux
xhost +local:docker
docker compose up -d sim
```

2. **Open a shell inside the container.** You can open as many as you like this way:

```
docker compose exec sim bash
```

3. **Build the AirMouse code** (needed once, and again after you change code):

```
am-build
```

4. **Launch the full simulation** (Gazebo, ArduPilot SITL, FAST-LIO, MAVROS and all AirMouse nodes). Wait about one minute until the log settles.

```
am-sim                                                  # with Gazebo window
ros2 launch airmouse_bringup sim.launch.py headless:=true   # faster, no window
```

5. **Open the dashboard** in a browser on your laptop: `http://localhost:8080`
6. **Press START.** The drone takes off, explores, tags survivors and lands at the exit by itself. Use **RECALL** to send it home early, or **ABORT / LAND** to land immediately.
7. **Stop** with Ctrl+C in the launch shell. To shut the container down: `docker compose down`.

Useful options

Option	Effect
maze:=maze_01	Which layout to load (name in airmouse_sim/mazes, or a path to a .txt file).
maze_scale:=1.3	Horizontal maze scale. Default 1.3 keeps the oversized sim drone in proportion; use 1.0 for exact competition dimensions.
headless:=true	No Gazebo window; uses less CPU/GPU.
am-start / am-abort	Send START / ABORT from a terminal instead of the dashboard.

Making your own maze

Copy `ros2_ws/src/airmouse_sim/mazes/maze_01.txt` and edit it. Each character is 0.5 m (times maze_scale): **#** wall, **.** free, **S** drone start (the drone faces up the page), **P**/**F**/**R** survivors (standing person, woman, sitting dummy). Then run `am-sim maze:=/airmouse/my_maze.txt`. The launch log prints the true survivor positions so you can check the results.

11. Using the ground control station

- **Top bar**: mission state, mission timer, flight mode, battery, SLAM health, explored area, number of survivors, and the latest status message.
- **Map (left)**: the live 2D map with the rescue grid (letters along the entry wall, numbers up the side), the drone (blue triangle), the planned path (dashed), the entry/exit (green square) and survivors (red dots with their grid cell; their cell is shaded).
- **Live camera (right)**: camera feed with person detections boxed in red.
- **Survivors table**: number, grid cell, position in metres and detection confidence.
- **Buttons**: START (only in IDLE), RECALL (only while exploring), ABORT / LAND (always; asks for confirmation). These are the only operator inputs, as the rules require.
- **Mission Planner** (optional): in sim connect TCP to 127.0.0.1 port 5762; on the drone listen on UDP 14550. Each confirmed survivor also appears there as a "SURVIVOR n at Xn" message.

12. Running on the real drone (Raspberry Pi 5)

> These steps are written but have **not been tested on hardware yet**. Expect to fix small issues the first time.

1. Flash **Raspberry Pi OS Lite 64-bit**, enable SSH, and install Docker (`curl -fsSL https://get.docker.com | sh`).
2. Wiring: Mid-360 Ethernet → Pi Ethernet (give the Pi the static IP 192.168.1.50/24); flight controller TELEM2 → Pi UART (GPIO 14/15, /dev/ttyAMA0, 921600 baud); USB camera → Pi USB; Pi powered from a 5 V / 5 A BEC.
3. Set the LiDAR IP (192.168.1.1XX, where XX = last two digits of its serial number) in `config/livox/MID360_config.json`.
4. Load `config/ardupilot/real.parm` into the flight controller with Mission Planner (Full Parameter List → Load from file → Write), then reboot it. Set battery voltages and RC switches for your build.
5. Measure and fill in every value marked **MEASURE** in `config/airmouse.yaml` (LiDAR and camera mounting, floor height, turn clearance).
6. On the Pi, build and start the robot container:

```
git clone https://github.com/harshkabir007/airmouse && cd airmouse
docker compose build robot      # about 1 hour on the Pi
docker compose up -d robot       # restarts automatically on boot
docker compose exec robot bash
am-build
yolo export model=/opt/models/yolo11n.pt format=ncnn imgsz=416   # faster detector
am-robot
```

7. Join the GCS laptop to the drone's Wi-Fi and open `http://<pi-ip>:8080`. Test with props off first, then tethered, before any free flight.

13. Configuration reference

All settings are in `ros2_ws/src/airmouse_bringup/config/`. **airmouse.yaml** is for the real drone; **sim.yaml** overrides it in simulation.

Parameter	Default	Meaning
arena.origin_x / origin_y	1.0 / 7.5 m	Where the rescue grid starts, relative to the drone start.
arena.cell_size, cols, rows	1.0 m, 15, 15	Rescue grid size.
commander.takeoff_alt	1.2 m	Flight altitude above the take-off point.
commander.inflation_radius	0.30 m (sim 0.42)	How far the drone centre stays from walls.
commander.turn_clearance	0.35 m (sim 0.48)	Minimum wall clearance before the drone may yaw.
commander.spin_at_goals	true	360° camera sweep at each frontier.
commander.max_explore_time_s	1260 s	Then return to the exit (30 min limit).
commander.hard_time_limit_s	1680 s	Land wherever the drone is.
commander.battery_return / land	30% / 18%	Battery levels for return and land.
survivor_detector.confidence	0.45	YOLO score threshold.
survivor_detector.confirm_hits	3	Sightings needed before a survivor is tagged.
pose_bridge.lidar_mount_*	inverted, 8 cm below	LiDAR position and orientation on the drone.
ArduPilot WPNAV_SPEED	70 cm/s	Cruise speed (sitl.parm / real.parm).

14. Troubleshooting

Symptom	What to do
Simulation stops by itself on Windows	Keep an Ubuntu window open while it runs.
Sim runs very slowly	Use the WSL compose override (software rendering). Use headless:=true. Close other heavy apps. Around 0.7× real time is normal on a laptop.
"service sim is not running"	Start it: docker compose -f compose.yaml -f docker/compose.wsl.yaml up -d sim
Dashboard says "waiting for SLAM"	Check: ros2 topic hz /Odometry. If it is empty, look for fast_lio errors in the launch log.
PreArm messages in the log while IDLE	Normal ("waiting for home"). The commander sets the EKF origin after you press START.
Drone lands with "SLAM lost" or "geofence"	A safety failsafe fired. Check the log just before it for the reason (often a collision).
Port 8080 or 5762 already in use	An old run is still alive: docker compose restart sim
Code changes have no effect	Run am-build again in the container, then relaunch.
Build fails on a new laptop	Check disk space and WSL memory (.wslconfig), then rebuild with docker compose build sim.

15. Repository layout

```
airmouse/
├─ compose.yaml                docker services: sim (laptop) and robot (Pi 5)
├─ docker/                     Dockerfile, WSL / NVIDIA overrides, shell env
├─ deps.repos                  pinned FAST-LIO and Livox driver versions
├─ scripts/                    build_deps.sh, build_ws.sh
├─ docs/                       implementation plan, this PDF and its generator
└─ ros2_ws/src/
   ├─ airmouse/                autonomy nodes (Python) + web/index.html
   │  └─ airmouse/             pose_bridge, mapper, commander, survivor_detector,
   │                           gcs_server, planning, arena, geometry, detectors
   ├─ airmouse_sim/            maze generator, drone sensor model, lidar_filter, mazes/
   └─ airmouse_bringup/        launch/ (sim, robot, core) and config/ (all parameters)
```

Working with git

```
cd ~/airmouse
git pull                        # get teammates' changes
git add -A && git commit -m "describe the change"
git push
```

16. Roadmap and next steps

Step	Goal
Simulation reliability	Run 10+ missions on several new layouts; aim for 6/6 survivors and no crashes. Add a batch script that runs headless missions and scores them.
Frontier retry	Retry skipped frontiers before returning, so no room is missed.
Speed	Cut exploration time: fewer revisits, skip spins in rooms already seen, raise cruise speed.
Smaller sim drone	Replace the oversized iris with a model matching the real airframe, then test at maze_scale 1.0.
GCS check	Visually test the dashboard and video in a browser; connect Mission Planner.
Hardware bench test	Build the robot image on the Pi; run the Mid-360 + FAST-LIO handheld; check CPU and temperature.
First flights	Tethered hover on the SLAM pose, then a mock corridor and room, then full missions.
Competition prep	Practise the 5-minute setup; confirm the grid convention with the organisers.

The full phased plan with exit criteria, hardware notes, rule-compliance checklist and risk list is in `docs/IMPLEMENTATION_PLAN.md`.

Appendix: command cheat sheet

```
# --- on the laptop, in Ubuntu (WSL) ---
cd ~/airmouse
docker compose -f compose.yaml -f docker/compose.wsl.yaml up -d sim   # start container
docker compose exec sim bash                                          # shell inside
docker compose restart sim                                            # kill a stuck run
docker compose down                                                   # stop everything

# --- inside the container ---
am-build                      # build AirMouse code
am-sim                        # run the simulation (add headless:=true)
am-start / am-abort           # mission start / land from a terminal
ros2 topic hz /Odometry       # is SLAM running?
ros2 topic echo /airmouse/status --once

# --- on the drone (Pi) ---
docker compose up -d robot && docker compose exec robot bash
am-build && am-robot
```